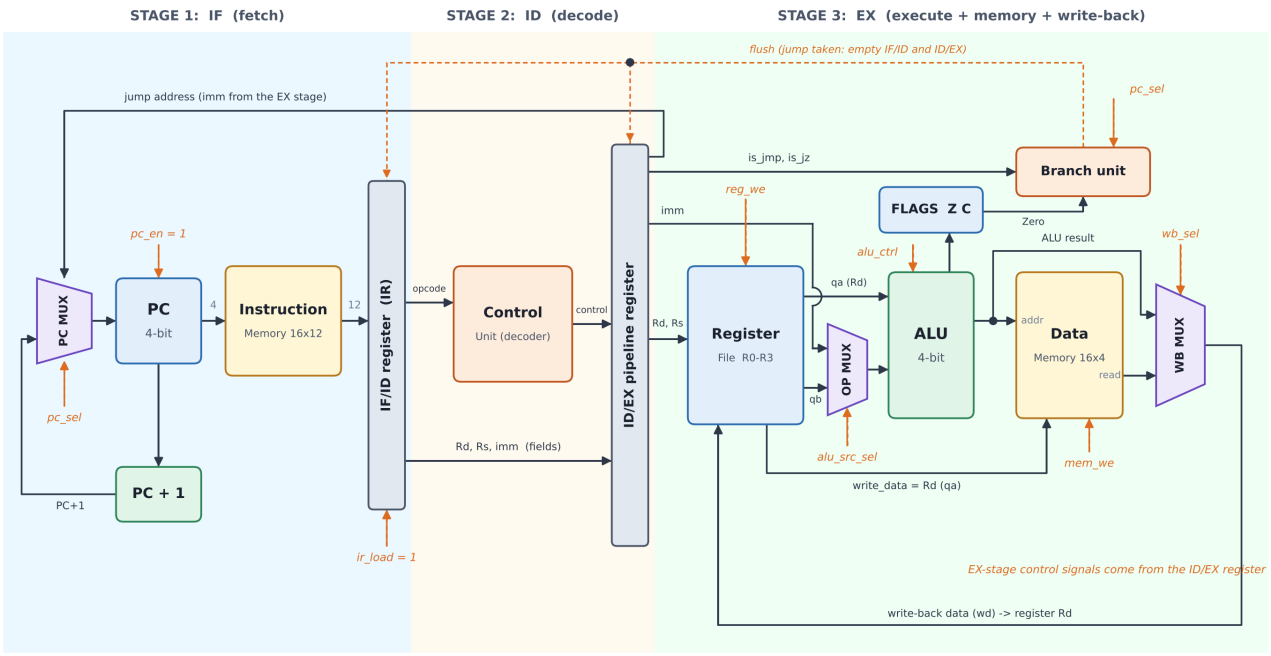
Code walkthrough

pipeline_register.v

ID/EX Pipeline Register

What it does	Carries the decoded instruction into EX
Where it works	between ID and EX
Size	93 lines, explained in 6 small parts
Modules in this file	pipeline_register
Calls (uses)	nothing (no other modules)
Called by	cpu_top.v - instance ID_EX

The second pipeline register. It stores everything the EX stage needs: all control signals made by the decoder, plus Rd, Rs, imm, the address and the valid bit. It is a plain bank of registers: inputs ending in _in are copied to the outputs at each clock edge.



The ID/EX pipeline register is the tall bar between ID and EX.

Map of the file

Part	Lines	What this part is
1	1-17	Header comment
2	18-21	Ports: clock, reset, flush
3	22-37	Ports: inputs from ID
4	38-54	Ports: outputs to EX
5	55-73	Reset or flush: make a bubble
6	74-93	Normal clock: pass everything forward

How to read this document: each part below shows a few lines of the real file, with their line numbers, followed by a plain-language explanation. Read the parts in order to understand the whole file without scrolling through it in one go.

Part 1 · Header comment (lines 1-17)

pipeline_register.v lines 1-17

```
 1  // ======================================================================
 2  // FILE: pipeline_register.v
 3  // The ID/EX PIPELINE REGISTER: it sits between the DECODE stage and
 4  // the EXECUTE stage.
 5  //
 6  // At every clock it copies everything the EXECUTE stage will need:
 7  //   - the control signals made by the decoder
 8  //   - the instruction fields: Rd, Rs, imm
 9  //   - the instruction address and a valid bit (for tracing)
10  //
11  // flush = 1 (taken jump) or rst = 1 loads a "do nothing" instruction:
12  // all control signals 0, so the EXECUTE stage changes nothing.
13  //
14  // CONNECTIONS
15  //   Calls       : nothing (no other modules)
16  //   Called by   : cpu_top.v - instance ID_EX
17  // ======================================================================
```

- Lists what is stored and explains that reset or flush loads a "do nothing" instruction.
- The **CONNECTIONS** lines at the end of the header list which other files this file uses (Calls) and which files use it (Called by).

Part 2 · Ports: clock, reset, flush (lines 18-21)

pipeline_register.v lines 18-21

```
18  module pipeline_register (
19      input             clk,
20      input             rst,
21      input             flush,
```

Same control inputs as the IR: clk, rst and flush.

Part 3 · Ports: inputs from ID (lines 22-37)

pipeline_register.v lines 22-37

```
22      // ---- inputs from the DECODE stage ----
23      input      [2:0]  alu_ctrl_in,
24      input             alu_src_sel_in,
25      input             wb_sel_in,
26      input             reg_we_in,
27      input             mem_we_in,
28      input             zero_we_in,
29      input             carry_we_in,
30      input             is_jmp_in,
31      input             is_jz_in,
32      input      [1:0]  rd_in,
33      input      [1:0]  rs_in,
34      input      [3:0]  imm_in,
35      input      [3:0]  pc_in,
36      input             valid_in,
37      input      [11:0] instr_in,
```

- Seven control signals from the decoder (alu_ctrl, alu_src_sel, wb_sel, reg_we, mem_we, zero_we, carry_we).
- Two jump markers (is_jmp, is_jz) for the branch unit.
- The instruction fields rd, rs, imm, plus pc, valid and the full instruction (for tracing).

Part 4 · Ports: outputs to EX (lines 38-54)

pipeline_register.v lines 38-54

```
38      // ---- outputs to the EXECUTE stage ----
39      output reg [2:0]  alu_ctrl,
40      output reg        alu_src_sel,
41      output reg        wb_sel,
42      output reg        reg_we,
43      output reg        mem_we,
44      output reg        zero_we,
45      output reg        carry_we,
46      output reg        is_jmp,
47      output reg        is_jz,
48      output reg [1:0]  rd,
49      output reg [1:0]  rs,
50      output reg [3:0]  imm,
51      output reg [3:0]  pc,
52      output reg        valid,
53      output reg [11:0] instr
54  );
```

The same signals without the _in ending. In cpu_top these get the prefix ex_ (ex_reg_we, ex_rd, ...) so you can see they belong to the EX stage.

Part 5 · Reset or flush: make a bubble (lines 55-73)

pipeline_register.v lines 55-73

```
55      always @(posedge clk) begin
56          if (rst || flush) begin
57              // ---- bubble: nothing will be written in EXECUTE ----
58              alu_ctrl    <= 3'd0;
59              alu_src_sel <= 1'b0;
60              wb_sel      <= 1'b0;
61              reg_we      <= 1'b0;
62              mem_we      <= 1'b0;
63              zero_we     <= 1'b0;
64              carry_we    <= 1'b0;
65              is_jmp      <= 1'b0;
66              is_jz       <= 1'b0;
67              rd          <= 2'd0;
68              rs          <= 2'd0;
69              imm         <= 4'd0;
70              pc          <= 4'd0;
71              valid       <= 1'b0;
72              instr       <= 12'hC00;
73          end
```

- On reset or flush every control signal becomes 0, valid becomes 0 and the instruction becomes C00.
- With all write enables at 0 the EX stage changes nothing: the flushed instruction disappears safely.

Part 6 · Normal clock: pass everything forward (lines 74-93)

pipeline_register.v lines 74-93

```
74          else begin
75              // ---- pass the decoded instruction on to EXECUTE ----
76              alu_ctrl    <= alu_ctrl_in;
77              alu_src_sel <= alu_src_sel_in;
78              wb_sel      <= wb_sel_in;
79              reg_we      <= reg_we_in;
80              mem_we      <= mem_we_in;
81              zero_we     <= zero_we_in;
82              carry_we    <= carry_we_in;
83              is_jmp      <= is_jmp_in;
84              is_jz       <= is_jz_in;
85              rd          <= rd_in;
86              rs          <= rs_in;
87              imm         <= imm_in;
88              pc          <= pc_in;
89              valid       <= valid_in;
90              instr       <= instr_in;
91          end
92      end
93  endmodule
```

Each output takes the value of its input. One clock later the EX stage executes exactly what the ID stage decoded.

> **In short:** A row of registers between ID and EX; flush turns its contents into a NOP.